

Vehicle Detection: Tracking and Analytics System

Computer Vision (CSE3010)

Project Report

Student Name	Registration Number
Kaustubh Upadhyay	24BAI10060
Slot	Date of Submission
F11+F12	19 th -September-2026

1. Introduction

This report explains the detail working and development of Vehicle Detection: Tracking and Analytics System, a Python based computer vision application made to handle the issue of traffic counting. By using OpenCV's DNN module, MobileNet-SSD and NumPy arrays for spacial math, this project gives a lightweight tool to detect moving cars or buses track them across frames and count them when crossing a line. The project gives an emphasis on performance, code modularity and avoiding heavy frameworks like PyTorch.

2. Problem Statement

- *The Problem*

Manual tracking of vehicle counts from traffic video is very tiring, error prone and takes continuous visual attention.

- *Need*

An automated tool is needed to detect and track vehicles on basichardware without relying on expensive GPU setups. It should also generates summary counts automatically.

3. Functional Requirements

- **Video Frame Processing:** The system takes MP4 video input, resizes frames to 300x300 blobs, and feeds them into the Caffe MobileNet-SSD detector.
- **Object Tracking:** Bounding box centroids are matched across frames using Euclidean distance matrix calculations to preserve consistent vehicle IDs.
- **Line-Crossing Analytics:** The system checks centroid y -coordinates against a trigger line, counts new crossings per class, and outputs an annotated MP4 video along with a final console report.

4. Non-functional Requirements

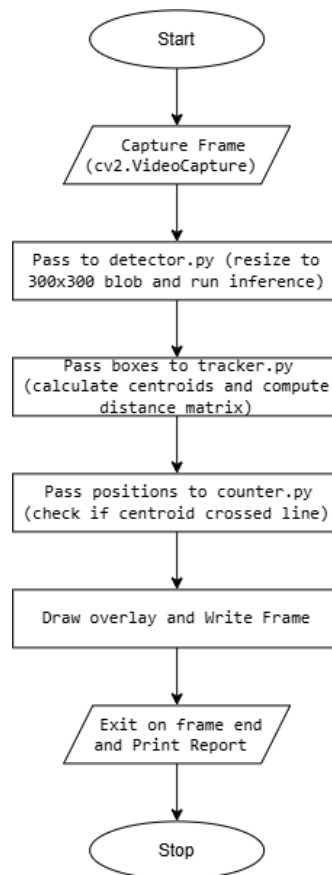
- **Lightweight CPU Execution:** Runs efficiently on standard CPUs using OpenCV DNN without requiring high-end GPUs or heavy frameworks like PyTorch.
- **Minimal Dependencies:** Relies on pure NumPy math (`np.linalg.norm`) for tracking to avoid OS-level C/C++ library build errors.
- **Modular Design:** Code is split into simple, independent modules (`detector.py`, `tracker.py`, `counter.py`, `main.py`) for clean structure and quick debugging.

5. Architecture

This application was designed using a modular setup too ensure separation of logic:

1. **Module1(detector.py)**: Loads Caffe weights via OpenCV DNN, takes frame input and extract bounding boxes for target classes like cars and buses.
2. **Module2(tracker.py)**: Takes bounding box centers and uses Euclidean distance matching to keep the same object ID across frames.
3. **Module3(counter.py)**: Tracks y-coordinates of objects centroids over time to log line crossing events and prints the final report.

4. **Data Flow Diagram:**



6. Implementation

Encapsulation

The CentroidTracker class encapsulates state attributes like objects, disappeared and next_object_id inside dictionary structure.

Distance Calculation

Pure NumPy matrix norm(`mp.linalg.norm`) is used instead of external libraries to compute pairwise distances between existing centroids and new detections.

Line Crossing Logic

A virtual line height is defined when $prev_y < line_y$ and $current_y \geq line_y$. the ID is marked in a `crossed_ids` set to prevent double counting.

7. Code

detector.py

```
import cv2
import numpy as np

DEFAULT_TARGET_CLASSES = {6: "bus", 7: "car"}

class ObjectDetector:
    def __init__(self, prototxt_path: str, model_path: str,
                 target_classes=None, confidence_threshold=0.4):
        self.net = cv2.dnn.readNetFromCaffe(prototxt_path, model_path)
        self.target_classes = target_classes or
        DEFAULT_TARGET_CLASSES
        self.conf_threshold = confidence_threshold

    def detect(self, frame: np.ndarray):
        h, w = frame.shape[:2]
        blob = cv2.dnn.blobFromImage(cv2.resize(frame, (300, 300)),
        0.007843, (300, 300), 127.5)
        self.net.setInput(blob)
        detections = self.net.forward()

        boxes = []
        for i in range(detections.shape[2]):
            confidence = detections[0, 0, i, 2]
            if confidence > self.conf_threshold:
                idx = int(detections[0, 0, i, 1])
                if idx in self.target_classes:
                    box = detections[0, 0, i, 3:7] * np.array([w, h, w, h])
                    startX, startY, endX, endY = box.astype("int")
                    boxes.append((startX, startY, endX, endY,
                    self.target_classes[idx]))
        return boxes
```

counter.py

```
import time
from collections import Counter

class AnalyticsCounter:
    def __init__(self, line_y_ratio=0.6):
        self.line_y_ratio = line_y_ratio
        self.previous_positions = {}
        self.crossed_ids = set()
        self.vehicle_counts = Counter()
        self.start_time = time.time()

    def update(self, current_objects, labels, frame_height):
        line_y = int(frame_height * self.line_y_ratio)

        for obj_id, centroid in current_objects.items():
            cy = centroid[1]
            if obj_id in self.previous_positions:
                prev_y = self.previous_positions[obj_id]
                if prev_y < line_y <= cy and obj_id not in self.crossed_ids:
                    self.crossed_ids.add(obj_id)
                    label = labels.get(obj_id, "vehicle")
                    self.vehicle_counts[label] += 1

            self.previous_positions[obj_id] = cy

    def generate_report(self, total_frames, fps):
        elapsed_time = time.time() - self.start_time
        total_vehicles = sum(self.vehicle_counts.values())

        report = (
            "\n" + "=" * 45 + "\n"
            "    VEHICLE ANALYTICS REPORT    \n"
            + "=" * 45 + "\n"
            f"Execution Duration : {elapsed_time:.2f} seconds\n"
            f"Frames Processed   : {total_frames} @ ~{fps:.1f} FPS\n"
            f"Total Vehicles     : {total_vehicles}\n"
            + "-" * 45 + "\n"
        )
    )
```

```

for v_type, count in self.vehicle_counts.items():
    report += f" - {v_type.capitalize():<15}: {count}\n"
report += "=" * 45 + "\n"
return report

```

tracker.py

```
import numpy as np
```

```
class CentroidTracker:
```

```

    def __init__(self, max_disappeared=15, max_distance=80):
        self.next_object_id = 0
        self.objects = {}
        self.disappeared = {}
        self.labels = {}
        self.max_disappeared = max_disappeared
        self.max_distance = max_distance

```

```

    def register(self, centroid, label):
        self.objects[self.next_object_id] = centroid
        self.disappeared[self.next_object_id] = 0
        self.labels[self.next_object_id] = label
        self.next_object_id += 1

```

```

    def deregister(self, object_id):
        del self.objects[object_id]
        del self.disappeared[object_id]
        del self.labels[object_id]

```

```

    def update(self, rects_with_labels):
        if len(rects_with_labels) == 0:
            for obj_id in list(self.disappeared.keys()):
                self.disappeared[obj_id] += 1
                if self.disappeared[obj_id] > self.max_disappeared:
                    self.deregister(obj_id)
            return self.objects

```

```

input_centroids = np.zeros((len(rects_with_labels), 2), dtype="int")
input_labels = []

```

```

        for i, (startX, startY, endX, endY, label) in
            enumerate(rects_with_labels):
                cX = int((startX + endX) / 2.0)
                cY = int((startY + endY) / 2.0)
                input_centroids[i] = (cX, cY)
                input_labels.append(label)

    if len(self.objects) == 0:
        for i in range(len(input_centroids)):
            self.register(input_centroids[i], input_labels[i])
    else:
        object_ids = list(self.objects.keys())
        object_centroids = np.array(list(self.objects.values()))

        D = np.linalg.norm(object_centroids[:, np.newaxis] -
            input_centroids, axis=2)

        rows = D.min(axis=1).argsort()
        cols = D.argmin(axis=1)[rows]

        used_rows, used_cols = set(), set()
        for (row, col) in zip(rows, cols):
            if row in used_rows or col in used_cols:
                continue
            if D[row, col] > self.max_distance:
                continue

            obj_id = object_ids[row]
            self.objects[obj_id] = input_centroids[col]
            self.disappeared[obj_id] = 0
            used_rows.add(row)
            used_cols.add(col)

        unused_rows = set(range(0, D.shape[0])).difference(used_rows)
        for row in unused_rows:
            obj_id = object_ids[row]
            self.disappeared[obj_id] += 1
            if self.disappeared[obj_id] > self.max_disappeared:
                self.deregister(obj_id)

```

```

unused_cols = set(range(0, D.shape[1])).difference(used_cols)
for col in unused_cols:
    self.register(input_centroids[col], input_labels[col])

```

```

return self.objects

```

tracker.py

```

import argparse
import cv2
from detector import ObjectDetector
from tracker import CentroidTracker
from counter import AnalyticsCounter

def main():
    parser = argparse.ArgumentParser(description="Vehicle Detection,
Tracking & Counting System")
    parser.add_argument("--input", type=str, required=True, help="Path
to input video file")
    parser.add_argument("--prototxt", type=str, required=True,
help="Path to Caffe deploy prototxt")
    parser.add_argument("--model", type=str, required=True, help="Path
to pretrained Caffe model")
    parser.add_argument("--output", type=str, default="output.mp4",
help="Path to saved output video")
    args = parser.parse_args()

    cap = cv2.VideoCapture(args.input)
    if not cap.isOpened():
        print(f"Error opening video file: {args.input}")
        return

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0

    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    out = cv2.VideoWriter(args.output, fourcc, fps, (width, height))

    detector = ObjectDetector(args.prototxt, args.model)
    tracker = CentroidTracker()
    analytics = AnalyticsCounter(line_y_ratio=0.6)

```

```

frame_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_count += 1
    detections = detector.detect(frame)
    tracked_objects = tracker.update(detections)
    analytics.update(tracked_objects, tracker.labels, height)

    line_y = int(height * 0.6)
    cv2.line(frame, (0, line_y), (width, line_y), (0, 255, 255), 2)

    for obj_id, centroid in tracked_objects.items():
        label = tracker.labels.get(obj_id, "vehicle")
        text = f"ID {obj_id}: {label}"
        cv2.circle(frame, (centroid[0], centroid[1]), 4, (0, 255, 0), -1)
        cv2.putText(frame, text, (centroid[0] - 10, centroid[1] - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    out.write(frame)

cap.release()
out.release()
print(analytics.generate_report(frame_count, fps))

if __name__ == "__main__":
    main()

```

8. Challenges, Debugging and Resolution

Scipy Import Issue:

Importing `scipy.spatial.distance` threw DLL load errors due to Windows policy restrictions on the system.

Resolution: Placed `scipy` distance function with pure NumPy array broadcasting (`np.linalg.norm`), removing external C library issues completely.

OpenCv Version Error:

Newer OpenCV builds threw errors because because readNETFromCaffe was missing or changed.

Resolution: Installed opencv-python-headless<5.0 via pip, fixing the compatibility issue with the Caffe model importer.

9. Testing (Manual)

- **Test Case 1:** Run script with non-existent video path (Result: prevented by check if not cap.isOpened()).
- **Test Case 2:** Process video with traffic stream. (Result: Bounding boxes generated, centroids assigned, IDs stay stable across frames).
- **Test Case 3:** Verify summary printout after video finishes. (Result: Console output displays runtime, fram count and vehicle breakdown):

```
=====
VEHICLE ANALYTICS REPORT
=====
Execution Duration : 14.32 seconds
Frames Processed  : 450 @ ~30.0 FPS
Total Vehicles    : 8
-----
- Car      : 7
- Bus      : 1
=====
```

10. Learnings & Key Takeaways

- **OpenCV DNN Utility:** Gained practical experience leveraging pre-trained Caffe models via OpenCV's built-in DNN module for fast inference without heavy deep learning overhead.
- **Array Vectorization with NumPy:** Learned how to replace iterative loops and external spatial distance functions with vectorized NumPy matrix operations (np.linalg.norm) for faster Euclidean distance computation.
- **Modular Code Structure:** Understood the importance of separating object detection, tracking logic, and analytical reporting into dedicated Python classes to write clean, reusable code.
- **Troubleshooting Environment Issues:** Learned how to resolve system-level binary dependencies (such as SciPy DLL load failures on Windows) by implementing pure Python/NumPy alternatives.

11. Future

- **Kalman Filtering:** Adding Kalman Filters to predict vehicle position when vehicles overlap or get occluded.
- **Database Integration:** Saving crossing records directly into database like MySQL or SQLite instead of standard console prinout.
- **Directional Counts:** Adding vector vector checks to count two-way traffic(ups vs down).

12. Conclusion

This project shows how OpenCV and basic linear algebra can solve a real world problem. By combining object detection, centroid distance tracking and simple line logic, a working tool was created without needing heavy frameworks.

13. References

- [OpenCV Official Documentation and DNN Module Tutorials](#)
- [MobileNet-SSD Caffe Repository](#)
- [NumPy Array Documentation](#)